

**Exercise 01 for MA-INF 2213 Advanced Computer Vision SS26**  
29.04.2026

**Important:** We recommend using **Python 3.12.9** for your solutions in this exercise. You are **not** allowed to use any Python modules besides NumPy for the parts that specifically state “using only NumPy library”. Otherwise, you will not get any points for those questions.

**Grading:** Submissions that generate runtime errors or produce invalid results (e.g. nans, inf, incoherent visualization) will not be graded. Also, use relative paths so your code is reproducible without requiring modifications.

**Plagiarism:** Plagiarism in any form is prohibited. If your solution contains code copied from any source, you will receive **0 points** for all exercise sheet.

## 1 Bounding Box Prediction

In this task, you are given a subset of The Oxford-IIIT Pet Dataset [1]. This subset includes images of cats and dogs. You will start with feature extraction. Thereafter, you are expected to build regression models from scratch with NumPy to learn prediction bounding box around the head of the animal in each image using training data. Then, you will optimize the model parameters by using validation data and evaluate their performance on the test data.

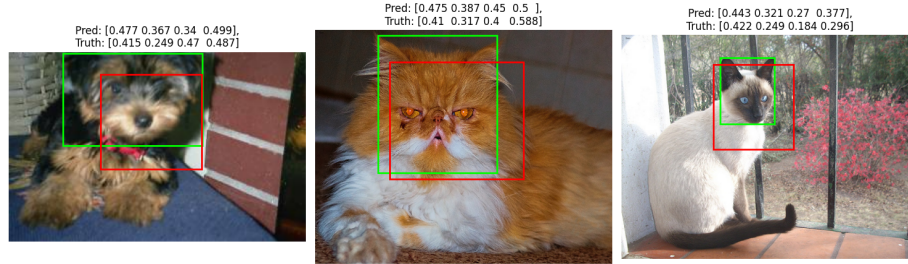


Figure 1: Visualizations of predicted bounding box around the head of the animal. While green box is ground truth, red box is predicted bounding box.

1. **Data Loading and Feature Extraction:** Instead of using the raw pixel values, you will use the Histogram-of-Gradients (HOG) method to extract features as a vector. Therefore, you should start by extracting HOG features and labels for the training, validation, and test splits separately. For this, you can use bounding box labels (starts with bbx-) will be obtained from the annotations file. You can also apply different conversions to labels: center format (center\_x, center\_y, width, height), normalized center format (scaled between 0 and 1 according the corresponding dimension), etc. After extraction, you should report the shape of each split.

*(2 Points)*

2. **Linear Regression:** Build a linear regression model for bounding box prediction using the closed-form solution. The regression model must be built using only **NumPy** library. Off-the-shelf models from other libraries (scikit-learn, etc.) are **not** allowed.

*(2 Points)*

3. **Gaussian Process Regression:** Build a Gaussian Process Regression (GPR) model using a

- linear kernel,
- polynomial kernel,
- and radial basis function (RBF) kernel,

for bounding box prediction. Your implementation should be aligned with the formulas in the lecture. Since the output is four-dimensional, you can fit one scalar GPR model per output dimension. The regression model must be built using only the **NumPy** library. Off-the-shelf models from other libraries (scikit-learn, etc.) are **not** allowed.

*(3 Points)*

4. **Optimization:** The main objective of this part is to get the best result with the GPR models you implemented in the previous part. Without tuning the hyperparameters of GPR models, the results will have low accuracy. For each kernel, you should find the best parameters via nonlinear optimization to maximize the marginal likelihood. You can optimize in log-space to enforce positivity. Please make sure you only use validation data to evaluate your model during these experiments. Once you have your best parameters, you can use test data to report your results. Then compare all the results and explain the differences in performance.

*(3 Points)*

5. **Evaluate Models:** Evaluate the performance of the each regression models on the test data with MSE (Mean Squared Error) and IoU (Intersection over Union) metrics. Evaluation function must be built using only the **NumPy** library. Off-the-shelf evaluation functions from other libraries (scikit-learn, etc.) are **not** allowed.

*(2 Points)*

6. **Visualization:** Visualize a random subset of your predictions for each model on the test set. Your visualization should include the original images, the predicted bounding boxes, and the ground truth bounding boxes as in Figure 1.

*(2 Points)*

7. **Bonus:** The best implementation in terms of IoU, which has done the model implementation and parameter optimization in a proper way, will get an extra 5 points.

## 2 Cat-Dog Classification

In this binary classification task, the same subset of The Oxford-IIIT Pet Dataset [1] will be used. You will predict whether the image shows a cat or a dog. You should use the same feature extraction pipeline as in the regression task. Adapt the label extraction for binary classification:

- If the species is cat, assign class label 0
- If the species is dog, assign class label 1

1. **Logistic Regression:** Build a non-linear logistic regress model with an RBF kernel from scratch using only NumPy. Off-the-shelf models from other libraries (scikit-learn, etc.) are **not** allowed. The loss function is the negative log-likelihood averaged over samples. Use gradient descent for optimization. The model must be built using only the **NumPy** library. Train your classifier on the training data. In these iterations, use the validation data to check your validation loss. You should plot training and validation loss curves using matplotlib and observe both curves to decide on learning rate and the number of iterations. You should save the best model weights based on validation loss throughout training and restore them at the end.

*(5 Points)*

2. **Evaluate Model:** Finally, evaluate the performance of the classifier on the test data with the accuracy metric. Accuracy is defined for binary classification as follows:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

where  $TP$ ,  $FP$ ,  $TN$ ,  $FN$  denote true positives, false positives, true negatives, and false negatives respectively. Use a decision threshold of 0.5 on the predicted probabilities. Evaluation function must be built using only the **NumPy** library. Off-the-shelf evaluation functions from other libraries (scikit-learn, etc.) are **not** allowed.

*(1 Points)*

## 3 Dataset Details

`dataset.zip` file consists of `images` folder and `annotations.csv` file. `images` folder includes 500 images for training set, 150 images for validation set, 150 images for test set. `annotations.csv` has following columns:

- **filename:** filename of image
- **split:** train, validation or test
- **species:** cat or dog

- `breed`: specific type of species
- `bbox_xmin`: left edge of bounding box
- `bbox_ymin`: top edge of bounding box
- `bbox_xmax`: right edge of bounding box
- `bbox_ymax`: bottom edge of bounding box
- `aspect_ratio`: width / height

According to this bbox representation, (0,0) point is the top left corner of images.

## References

- [1] O. M. Parkhi, A. Vedaldi, A. Zisserman and C. V. Jawahar, "Cats and dogs," 2012 IEEE Conference on Computer Vision and Pattern Recognition, Providence, RI, USA, 2012, pp. 3498-3505